Agent Zoo — Technical Implementation Report

Engineering reference for the agents in `opposition-agents-playing-mtg`

Abstract

This document is the engineering counterpart to the scientific writeup *Opposition Agents Playing Magic: The Gathering* (`paper/opposition_agents_mtg.tex`). Where the paper focuses on *objectives*, this report focuses on *exactly what the code does*: class signatures, control flow, configuration tables, failure modes, and the structured reasoning traces each agent emits. All cited paths are workspace-relative.

Contents

1 Theoretical background (1 page)

The framework treats Magic: The Gathering (MTG) as a partially-observable Markov game. Each player's perspective is a POMDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, T, \Omega, R, \gamma)$ with hidden state (opponent hands and library order). Concretely:

- $\mathcal{S}$: full `GameState` including all players' zones, the stack, mana pools, life totals, and bookkeeping flags. Conservatively $|\mathcal{S}| > 10^{800}$.
- $\mathcal{O}$: seat-filtered snapshot emitted by phase-rs `StateUpdate`, then projected into our policy view by `src/integrations/phase_rs/adapter.py`.
- $\mathcal{A}$: legal action list from phase-rs wire messages, mapped into internal `Action` values by the same adapter, typically $|\mathcal{A}| \in [1, 60]$.
- T: phase-rs runtime transition system (Rust engine), deterministic given the random seed and chosen actions.
- R: terminal ± 1 on win/loss; agents may shape via Δlife, Δcards, Δtempo when training a value head.

Three families of policies. The agents implement:

1. *Reactive* (Random, Heuristic, KG-Heuristic, LLM): map $o_t \to a_t$ directly, no internal state.
2. *Model-based* (World-Model): learn $\hat{T}$ in latent space and pick $\arg\max_a \mathbb{E}_{\hat{T}}[V(s_{t+H})]$.
3. *Belief-driven* (Active-Inference, LLM-Fusion): maintain a belief b_t over hidden state and minimise expected free energy $G(\pi) = \underbrace{-\mathbb{E}_q[R]}_{\text{pragmatic}} + \underbrace{-\mathbb{E}_q[\mathrm{KL}(b_{t+1} \| b_t)]}_{\text{epistemic}}$.

A self-contained derivation is in §9. See also the corresponding section in the companion paper.

The same formal stack also supports a collective-intelligence workflow: different agent families generate trajectories, those trajectories are mined into append-only KG evidence with provenance, and later agents use that shared graph memory as an additional prior. This separates immutable card facts from learned strategic evidence while allowing cumulative cross-agent learning across runs.

2 The contract every agent satisfies

Every agent class implements a single method:

```python
class MTGAgent(Protocol):
    name: str

    async def decide_action(
        self,
        game_state: GameState,
        legal_actions: list[Action],
    ) -> Action: ...
```

Agents must *never* mutate `game_state`; only the engine is allowed to. An agent that returns an action not in `legal_actions` is treated as a forfeit by the phase-rs integration runner.

Determinism: any RNG owned by an agent must be a `random.Random` instance constructed from the seed passed by the runner. This guarantees `(seed, decklists, agent_types)` replays bit-exactly.

3 The lineup

Agent	**Module**	**Strategy**
Random	`random_agent.py`	Uniform over legal actions.
Heuristic	`heuristic_agent.py`	Hand-tuned action priorities.
KG-Heuristic	`kg_heuristic_agent.py`	Heuristic + Neo4j combo lookup.
Ollama (LLM)	`ollama_agent.py`	Local LLM picks an action index.
Active-Inference	`active_inference_agent.py`	Bayesian belief, EFE minimisation.
World-Model	`world_model_agent.py`	V+M+C latent rollouts.
LLM-Fusion	`llm_fusion_agent.py`	Weighted vote of all four signals.
Human	`human_agent.py`	Stdin prompt, used for debugging.

4 Glossary and symbols

Symbol / term	**Expanded form**	**Meaning in this repository**
$POMDP$	Partially Observable Markov Decision Process	Formal model of each seat's decision process under hidden information.
V, M, C	Encoder, Dynamics, Controller	World-model split: latent encoder, recurrent transition model, and policy/value heads.
$JEPA$	Joint Embedding Predictive Architecture	Predictor head trained in latent space without reconstruction.
EFE / $G(\pi)$	Expected Free Energy	Active-inference objective combining pragmatic and epistemic terms.
KG	Knowledge Graph	Neo4j-backed symbolic substrate over cards, combos, mechanics, and archetypes.
$APNAP$	Active Player, Non-Active Player	Priority ordering used by the stack and trigger resolution paths.
CR	Comprehensive Rules	Canonical MTG rules text used as implementation reference for mechanics.
$\mathcal{A}_{legal}$	Legal action set	Engine-generated action list; agents must choose exactly one member.
$\mathcal{D}$	Replay buffer	Collected trajectories (s_t, a_t, r_t) for model training and analysis.

5 Per-agent reference

5.1 RandomAgent

File: `src/agents/random_agent.py`. **Purpose:** baseline / smoke test.

```
def decide_action(self, game_state, legal_actions):
    return self._rng.choice(legal_actions)
```

Configuration: `seed: int` (forwarded to `random.Random`). Failure modes: none. Deterministic given a seed.

5.2 HeuristicAgent

File: `src/agents/heuristic_agent.py`.

Decision flow.

1: scored $\leftarrow$ [(priority(a), $-$tiebreak(a), a) for $a \in$ legal_actions]
2: scored.sort(descending)
3: **return** scored[0].a

Priority table (`ACTION_PRIORITIES`):

Action type	Priority
PLAY_LAND	100
CAST_SPELL	90
DECLARE_ATTACKERS	80
DECLARE_BLOCKERS	70
ACTIVATE_ABILITY	60
PASS_PRIORITY	1
CONCEDE	0

Tiebreaks: prefer cheaper spells, prefer creatures with higher power when attacking. Failure modes: never holds up mana for instants (known limitation).

5.3 KGHeuristicAgent

File: `src/agents/kg_heuristic_agent.py`.

Wraps `HeuristicAgent` but rescores `CAST_SPELL` actions using the Neo4j graph through `src/knowledge/kg_query.py:score_card_for_state`:

$$\text{kg\_score}(c) = 10 \cdot \mathbf{1}[c \in \text{NearComboMissingPieces}(s)] + \sum_{p \in \text{permanents}(s)} \text{strength}(\text{KG-edge}(c, p)). \tag{1}$$

The combined score is $\text{prio}(a) + \lambda \cdot \text{kg\_score}(c)$ with λ=5. Configuration: `neo4j_uri`, `auth`, `enable_combo_lookup` (bool). Failure mode: if Neo4j is unreachable, falls back silently to plain HeuristicAgent.

5.4 OllamaAgent (and LLMAgent alias)

File: `src/agents/ollama_agent.py`.

Builds a textual board state via `format_state_for_llm()`, sends a chat completion request to `http://localhost:11434/api/chat` (model `gemma4:e2b`, temperature 0.2, `keep_alive=30m`, timeout 120s), and parses an integer action index out of the reply.

```
prompt = (
    "You are an MTG expert. Choose the BEST action by index.\n"
    f"State:\n{format_state_for_llm(state)}\n"
    f"Legal actions:\n{enumerate_actions(legal_actions)}\n"
    "Respond with ONLY the index."
)
resp = requests.post(OLLAMA_URL, json={"model": MODEL, "messages": [...]})
idx = int(re.search(r"\d+", resp.json()["message"]["content"]).group())
return legal_actions[idx % len(legal_actions)]
```

Failure modes: timeout → fall back to HeuristicAgent; non-integer reply → fall back to RandomAgent over a small legal-action subset.

5.5 ActiveInferenceAgent

File: `src/agents/active_inference_agent.py`.

Maintains a Dirichlet belief over the opponent's hand composition and minimises expected free energy:

$$G(\pi) \;=\; -\,\mathbb{E}_{q(s'|\pi)}[R(s')] \;-\; \mathrm{KL}\big[q(s'|\pi)\,\|\,b_t\big]\,. \tag{2}$$

The first term is the pragmatic value (favouring high-utility outcomes), the second is the epistemic value (favouring information gain). Belief update on each opponent action uses Bayes' rule with a card-likelihood table seeded from the decklist (see `docs/OPPONENT_MODELING_WITH_DECKLIST.md`).

5.6 WorldModelAgent

File: `src/agents/world_model_agent.py`. **Backbone:** `src/world_model/world_model.py`.

Training now uses the external `stable-pretraining` orchestration layer and the surrounding `stable-worldmodel` research stack rather than a bespoke in-repo trainer. Concretely, `scripts/train_stable_worldmodel.py` wraps the local MTG `WorldModel` in a `stable_pretraining.Module` and drives optimisation through `stable_pretraining.Manager`; this mirrors the software split used by LeWorldModel, which keeps the architecture and loss local while delegating training/runtime plumbing to the upstream libraries [1, 2, 3].

Components (V+M+C, after Ha & Schmidhuber).

$$\begin{aligned}
z_t &= \mathrm{Enc}_V(o_t) && \text{(perceptual encoder, 768-d)}\\
g_t &= \mathrm{Enc}_{KG}(\mathcal{G}, s_t) && \text{(GraphSAGE, 128-d)}\\
(h_t, c_t) &= \mathrm{LSTM}_M([z_t; g_t], (h_{t-1}, c_{t-1})) &&\\
\hat{r}_t &= V(h_t) && \text{(value head)}\\
\pi(a|h_t) &= \mathrm{softmax}(C(h_t)) && \text{(controller)}
\end{aligned}$$

Decision rule. For each candidate action $a \in \mathcal{A}$, roll out K trajectories of horizon H in latent space (no actual rules-engine simulation):

$$\mathrm{score}(a) = \frac{1}{K}\sum_{k=1}^{K} V(h^{(k)}_{t+H} \mid a_t = a). \tag{3}$$

Defaults: $K = 8$, $H = 10$, sampling temperature 1.15.

Checkpoint loading. `WorldModel.load` now tolerates three checkpoint shapes (canonical, JEPA-only, and free-form), warning-and-continuing rather than raising. The agent factory (`src/agents/__init__.py:_make_world_model`) falls back to a default-initialised network if the checkpoint file does not exist, so the harness can be smoke-tested before training.

Checkpoint troubleshooting.

- Shape mismatch in controller head: occurs when $|\mathcal{A}_{legal}|$ conventions differ between checkpoint and runtime. Re-export checkpoint after rebuilding action encoding.
- Missing JEPA keys: expected when loading pre-JEPA checkpoints; loader falls back to random init for missing tensors and logs a warning.
- KG context mismatch: if graph feature width changed, pass a matching checkpoint or train one epoch with `-resume` to adapt projection layers.

5.7 LLMFusionAgent

File: `src/agents/llm_fusion_agent.py`.

Combines four signals into a single ranking:

$$\text{score}(a) = \alpha_h \, s_{\text{heur}}(a) + \alpha_g \, s_{\text{KG}}(a) + \alpha_w \, s_{\text{WM}}(a) + \alpha_\ell \, \log p_{\text{LLM}}(a), \tag{4}$$

with default weights $(\alpha_h, \alpha_g, \alpha_w, \alpha_\ell) = (0.15, 0.15, 0.40, 0.30)$ from `FusionConfig`. When any signal exceeds the `obvious_threshold` (0.8), it short-circuits the vote to save an LLM call.

Algorithm 1 LLM-Fusion turn (simplified)

Require: state s_t, legal actions $\mathcal{A}$, weights α

$S_h \leftarrow \text{HeuristicAgent.score\_all}(s_t, \mathcal{A})$
$S_g \leftarrow \text{KG.score\_all}(s_t, \mathcal{A})$
if $\max_a S_h(a) > \tau_{\text{obvious}}$ **then return** $\arg\max_a S_h(a)$
end if
$S_w \leftarrow \text{WM.value\_rollout}(s_t, \mathcal{A})$
$\mathcal{A}^* \leftarrow$ top-N actions by S_w
$S_\ell(a) \leftarrow \log p_{\text{LLM}}(a \mid \text{prompt}(s_t, \mathcal{A}^*))$
return $\arg\max_{a \in \mathcal{A}^*} \alpha_h S_h(a) + \alpha_g S_g(a) + \alpha_w S_w(a) + \alpha_\ell S_\ell(a)$

FusionConfig defaults.

Parameter	Default	Role
`heuristic_weight`	0.15	Rules-aware tactical prior
`kg_weight`	0.15	Combo and synergy prior from Neo4j
`wm_weight`	0.40	Latent rollout value estimate
`llm_weight`	0.30	Language prior over legal indices
`top_n`	5	Candidate set handed to LLM stage
`obvious_threshold`	0.8	Short-circuit threshold for cheap decisions

One-turn message flow (ASCII).

```
priority_loop -> LLMFusionAgent.decide_action(state, legal_actions)
     LLMFusionAgent -> HeuristicAgent.score_all: S_h
     LLMFusionAgent -> KG query layer: S_g
     alt max(S_h) > obvious_threshold
          LLMFusionAgent --> priority_loop: argmax(S_h)
     else
          LLMFusionAgent -> WorldModelAgent.rollout_values: S_w
          LLMFusionAgent -> OllamaAgent.log_probs(topN by S_w): S_l
          LLMFusionAgent -> fuse(alpha_h*S_h + alpha_g*S_g + alpha_w*S_w + alpha_l*S_l)
          LLMFusionAgent --> priority_loop: chosen legal action
     end
priority_loop -> engine: apply chosen action
```

5.8 HumanAgent

File: `src/agents/human_agent.py`. Renders the legal actions to stdout and reads a 1-based index. Used by `examples/play_logged_game.py` for hand-debugging pathological states.

6 Debugging recipes

6.1 Agent loops on pass priority

Symptoms: repeated `PASS_PRIORITY` despite non-trivial board states.
Checklist:

- Inspect `legal_actions` logs from `src/orchestrator/priority_loop.py`; if only pass/concede exist, the issue is engine-side rather than policy-side.
- Verify stack resolution path in `src/engine/stack.py` and trigger enqueueing in `src/engine/triggered_abil`
- Confirm current phase allows the expected action class (main phase vs combat priority windows).

6.2 Agent never attacks

Symptoms: board develops, but attackers are never declared.
Checklist:

- Check `src/engine/combat.py:legal_attackers` output for summoning-sickness and tap-state filtering.
- Ensure power/toughness modifiers from equipment and temporary effects reach `effective_power`/`effective_`
- For heuristic agents, inspect `ACTION_PRIORITIES` and tiebreak functions for accidental over-penalisation of combat lines.

6.3 World-model agent regresses after checkpoint swap

Symptoms: sudden Elo drop or random-looking actions after loading a new checkpoint.
Checklist:

- Confirm checkpoint compatibility warnings from `WorldModel.load`; missing keys should be expected and logged.
- Run a deterministic smoke pod (seed fixed) and compare trace-level `wm` scores before/after swap.
- Recompute normalisation statistics if latent feature scales changed between training runs.

6.4 LLM fusion hangs or times out

Symptoms: long think times or fallback-only behaviour.
Checklist:

- Verify Ollama endpoint and model are available (see `docs/OLLAMA_SETUP.md`).
- Lower `top_n` and increase `obvious_threshold` to reduce LLM calls under load.
- Confirm timeout fallback path returns a legal action and records the fallback reason in reasoning traces.

7 Reasoning trace contract

Every agent except Random/Human emits a structured JSON record on each decision into `state.log_jsonl(...)`:

```
{
  "turn": 7, "phase": "main1", "agent": "llm_fusion",
  "chosen_action_idx": 3,
  "scores": {"heuristic": [...], "kg": [...], "wm": [...], "llm": [...]},
  "weights": {"heur": 0.15, "kg": 0.15, "wm": 0.40, "llm": 0.30},
  "elapsed_ms": 412,
  "explanation": "WM rollout favours T+8 board state with..."
}
```

Trace files land at `runs/<exp>/game_NNN.jsonl` alongside the human-readable `game_NNN.log`.

Minimal deterministic trace example (heuristic).

```
{
     "turn": 3,
     "phase": "main1",
     "agent": "heuristic",
     "chosen_action_idx": 1,
     "scores": {"heuristic": [100.0, 90.0, 1.0]},
     "weights": {"heur": 1.0},
     "elapsed_ms": 1,
     "explanation": "play land has highest static priority"
}
```

Fusion trace example (all channels active).

```
{
     "turn": 7,
     "phase": "main1",
     "agent": "llm_fusion",
     "chosen_action_idx": 3,
     "scores": {
          "heuristic": [0.32, 0.81, 0.11, 0.44],
          "kg": [0.05, 0.10, 0.62, 0.18],
          "wm": [0.40, 0.21, 0.73, 0.68],
          "llm": [-1.50, -0.20, -1.10, -0.35]
     },
     "weights": {"heur": 0.15, "kg": 0.15, "wm": 0.40, "llm": 0.30},
     "elapsed_ms": 412,
     "explanation": "wm+llm agree on index 3 after top-N pruning"
}
```

8 Test pointers

Agent	Test file
RandomAgent	`tests/test_random_agent.py`
HeuristicAgent	`tests/test_heuristic_agent.py`
KGHeuristicAgent	`tests/test_kg_heuristic_agent.py`
OllamaAgent (mocked)	`tests/test_ollama_agent.py`
ActiveInferenceAgent	`tests/test_active_inference_agent.py`
WorldModelAgent	`tests/test_world_model_agent.py`
LLMFusionAgent	`tests/test_llm_fusion_agent.py`

Smoke commands.

```
.\.venv\Scripts\python.exe -m pytest tests/ -q
.\.venv\Scripts\python.exe examples\play_edh_pod.py --max-turns 6 --seed 7 --model none
scripts\run_overnight_benchmarks.bat --quick
```

9 Theoretical foundations (extended)

9.1 POMDPs and partial observability

A finite-horizon POMDP is the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma, b_0)$. The optimal value function over beliefs satisfies

$$V^*(b) = \max_a \Big[\sum_s b(s) R(s,a) + \gamma \sum_o \Pr(o \mid b, a)\, V^*(b'_o) \Big], \tag{5}$$

where b'_o is the Bayes update of b after action a and observation o. Exact solution is intractable for $|\mathcal{S}| > 10^{800}$; all our policies are approximations.

9.2 World models as latent imagination

Following Ha & Schmidhuber and Hafner et al. (Dreamer), we factor the agent as V (encoder), M (recurrent dynamics), and C (controller). Training the dynamics jointly minimises

$$\mathcal{L}_{\text{dyn}} = \sum_t \big\| \hat{z}_{t+1} - \text{sg}(z_{t+1}) \big\|_2^2 + \beta \cdot \text{KL}(q(z_t|o_t) \,\|\, p(z_t|h_{t-1})), \tag{6}$$

where sg is stop-gradient. We follow the JEPA recipe of LeCun & Assran: predict in representation space, not in pixel space. The single tunable knob is β (`-jepa-beta`, default 1.0). In the current repository this loss is still computed by the local MTG model, but the optimisation loop, callback stack, and checkpoint orchestration are delegated to `stable-pretraining`; `stable-worldmodel` remains the upstream runtime substrate around data collection and world-model evaluation conventions [1, 2].

9.3 Free energy and active inference

Friston's free-energy principle frames perception and action as joint inference. The variational free energy is

$$F[q] = \underbrace{\mathbb{E}_q[\log q(s) - \log p(o,s)]}_{\text{evidence bound}} = \text{KL}(q \| p(s|o)) - \log p(o). \tag{7}$$

Action selection minimises *expected* free energy, which decomposes as

$$G(\pi) = \underbrace{-\mathbb{E}_q[\log p(o \mid C)]}_{\text{pragmatic}} + \underbrace{-\mathbb{E}_q[\text{KL}(q(s \mid o, \pi) \,\|\, q(s \mid \pi))]}_{\text{epistemic}}. \tag{8}$$

The pragmatic term encodes goal-seeking; the epistemic term encodes information-gathering. Bayesian RL is recovered when the prior C is uniform, and pure exploration is recovered when rewards are flat.

9.4 Knowledge-graph grounding

Cards form an inductive graph $\mathcal{G} = (V, E)$ with edges for *combo*, *synergy*, *counters*, and *enables* (see `src/knowledge/`). GraphSAGE produces inductive node embeddings $\mathbf{h}_v \in \mathbb{R}^{128}$ that are concatenated with the perceptual latent z_t before the recurrent dynamics. This is the V+M+C model with an additional *graph* channel Enc_{KG}.

9.5 LLM-as-policy under legal-action constraints

The LLM never freely generates text. Given the prompt $\rho(s_t, \mathcal{A})$, we read off

$$\log p_{\text{LLM}}(a \mid s_t) = \text{LM}(\text{idx}(a) \mid \rho(s_t, \mathcal{A})) \tag{9}$$

and mask any index outside $\{0, \ldots, |\mathcal{A}| - 1\}$. This is the standard "constrained decoding" trick and prevents illegal hallucinated moves.

9.6 Self-play promotion as fixed-point iteration

The iterative training loop in `scripts/train_pipeline.py` implements a champion-vs-challenger schema: a challenger trained on the latest replay buffer plays a best-of-N against the current champion; promotion happens iff the win rate exceeds 0.55. This is a stochastic-approximation fixed-point on the policy improvement operator and converges (in expectation) provided the dynamics model is locally accurate.

10 Where to look next

- Companion paper: `paper/opposition_agents_mtg.tex`.
- Single source of truth: `IMPLEMENTATION_PLAN.md`.
- Engine docs: `ARCHITECTURE.md`, `DEVELOPMENT.md`.
- World-model design: `docs/WORLD_MODEL_DESIGN.md`.
- Opponent modelling: `docs/OPPONENT_MODELING_WITH_DECKLIST.md`.

11 References

References

[1] R. Balestriero, H. Van Assel, S. BuGhanem, and L. Maes. stable-pretraining-v1: Foundation Model Research Made Simple. *arXiv:2511.19484*, 2025. Software: `https://github.com/galilai-group/stable-pretraining`.

[2] L. Maes, Q. Le Lidec, D. Haramati, N. Massaudi, D. Scieur, Y. LeCun, and R. Balestriero. stable-worldmodel-v1: Reproducible World Modeling Research and Evaluation. *arXiv:2602.08968*, 2026. Software: `https://github.com/galilai-group/stable-worldmodel`.

[3] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv preprint*, 2026. Software: `https://github.com/lucas-maes/le-wm`.